PA1: Asteroids

CIIC-4010/ICOM4015

DUE: October 16, 2026

1 Description

Welcome to the first official project of the semester! From now on projects will consist of working on partially complete videogames. For this project, you will be working on an implementation of the game *Asteroids*; you will be required to work with your partner to fix bugs and add features to the existing structure. Before starting, make the most of these projects as they can help you study for future labs and/or tests, but most importantly, have fun while working on these projects, enjoy the process, and feel free to make any additions you'd like to the project.

2 General Instructions

Projects will be worked in teams of **two students** and will be hosted on GitHub Classroom. Personal repositories will not be graded. Just like in your previous projects, one partner will create the team repository while the other joins it. For this reason, it is best to wait until you have a partner to make a team. If you have not found a partner within a week of the project's release, notify a staff member as quickly as possible! Remember you can repeat the same partner you had for PA0 but NOT your partner from PA1. Use skills and best practices learned in the Git & GitHub lab, such as using branches for every feature. **NOTE: Include your and your partner's name and email in the README.md file. Each partner in a team must commit their contributions using their individual GitHub credentials.**

3 Teamwork

A big emphasis of the projects is teamwork. You will find that throughout your career, you will have to learn to work with different people in order to accomplish great things. With that in mind, here are some things to remember: • Read the requirements and either follow the suggested timeline or make one that works better for you.

- Read through the requirements and delegate individual features between you and your partner in an equitable way.

- Feel free to collaborate on individual features with your partner, but do not do their work.
- Meet frequently to share progress and readjust personal deadlines or redelegate tasks if needed. Make sure that both you and your partner are making progress. Notify the staff and ask for help is this is not the case. • Make sure to work on one spec per branch and never work directly on the master branch.
- Push to GitHub only your master branches.
- Use this opportunity to network with your partner. They may be a future coworker or job reference, so it is best to maintain professionalism. For instance, be responsive to your partner's messages and make yourself available to meet when needed. • If you encounter any issues contact a staff member as soon as possible. This may include any major problem with your partner or a failure to progress in the project.

3.1 Problems with partner

If you encounter any problems with your partner, such as a lack of progress from their part or feeling like the workload is unfair please follow these steps: • Arrange a one-on-one meeting with your partner and try to work out the specs between the two of you. Some issues may be attributed to miscommunication and can oftentimes be solved by talking. • If your partner will not meet or you cannot reach an agreement by talking, contact a staff member **immediately**.

4 Requirements

As part of your assignment, you must implement the following specs. For full credit, your team must develop these new features while avoiding code redundancy and duplication, following the base structure of the project, and making the least amount of alterations to the source code as possible. You should also strive to keep your code documented and legible. That is to say, use meaningful variable and method/function names and if a part of your implementation isn't obvious, feel free add a comment explaining it. For your convenience, the project has been divided into three phases with suggested deadlines. There is no penalty for not meeting any deadlines, except for the final due date on **October 16**, but they are a good way of knowing if you are on schedule.

4.1 Joining GitHub Assignment

Please fill out this form with you and your partner's name to accept the assignment (PA1 Form) Do not create a team unless you have a partner, only one member has to fill out this form. Again, if you haven't found a partner by **October 3,** contact a member of the staff as soon as possible!

4.2 Phase 1 Spaceship Malfunctions (Suggested Deadline October 3)

- **Starting Lives:** When the game begins, the player must start with exactly **3 lives**. The number of remaining lives should be displayed on the screen so the player can keep track of their current health. Whenever the spaceship is destroyed, the player loses one life. If the player loses all three lives, the game should recognize that there are no lives remaining and transition to the appropriate game-over state.

- **Bullets Don't Work:** During Phase 1, the spaceship's shooting system isn't functional. Although Spacebar is presented as the shooting key, pressing it won't fire a bullet. Implement the shooting bullet key.

- **Spaceship Movement:** The player will control the spaceship using the **W, A, S, and D keys**. The W key will move the spaceship upward, A will move it to the left, S will move it downward, and D will move it to the right. The spaceship should respond continuously while the corresponding key is held down. **The spaceship currently moves with arrow keys.**

4.3 Phase 2 More Repairs (Suggested Deadline October 10)

- **Asteroid Scoring:** The game will include a scoring system based on the size of the asteroid destroyed by the player. Destroying a **large asteroid** will award **250 points**, destroying a **medium asteroid** will award **100 points**, and destroying a **small asteroid** will award **50 points**. The player's score should be displayed on the screen and updated immediately whenever an asteroid is successfully destroyed, with the amount awarded corresponding to the asteroid's size.

- **Pause Button:** The game will include a pause function that allows the player to temporarily stop the game and resume it later. When the game is paused, all gameplay activity must stop, including the movement of the spaceship, asteroids, bullets, collisions, and other game elements. When the player resumes the game, everything should continue from the exact state in which it was paused rather than restarting or resetting the game. Restarting the

game is a separate action that should reset the current game and begin again from its starting conditions.

- **Red Icon When on Last Life:** The game will display a red warning icon when the player reaches their final remaining life. The icon should not appear while the player has more than one life. Once the player has only one life remaining, the red icon should become visible to clearly warn the player that they are close to game over. The icon should remain visible while the player has one life and should disappear if the player gains another life.

4.4 Phase 3 Adding Gadgets (Suggested Deadline October 16)

- **Bonus Lives:** The player will be able to earn additional lives by reaching score milestones. For every 1,000 points earned, the player will receive one bonus life, as long as they currently have fewer than 6 lives. The player can never have more than 6 lives at one time. For example, reaching 1,000 points can increase the player's lives from 3 to 4, reaching 2,000 points can increase them to 5, and reaching 3,000 points can increase them to 6. Once the player has reached the maximum of 6 lives, additional 1,000-point milestones cannot increase the life count beyond 6.

- **Spaceship Color:** The spaceship will change color based on the player's score, with a new color being applied every **500 points**. The required color sequence is **Blue, Green, Red**, and this sequence will repeat throughout the game. For example, the spaceship will become blue when the player reaches 500 points, green at 1,000 points, and red at 1,500 points. The color change is purely visual and should not affect the spaceship's movement, lives, shooting, or other gameplay mechanics.

4.5 Bonus!

Feel free to add any bonus content as you'd like! However, this bonus should **not** change the game's functionality, **which can prevent other features from being graded**. Here are some examples of what a bonus can be!

- **New Textures or Sounds**
- **Themes**
- **Thruster boost**
- **New Game Modes**

Happy Coding!